

INSTALL & RUN

```
uv add "fastapi[standard]" # or pip install
from fastapi import FastAPI
app = FastAPI()

@app.get("/")
async def root() -> dict[str, str]:
    return {"hello": "world"}

fastapi dev # reload, 127.0.0.1 (main.py)
fastapi run # production, 0.0.0.0

/docs Swagger UI /redoc ReDoc /openapi.json
schema. Still 0.x: pin the version.
```

PATH OPERATIONS

```
@app.get | post | put | patch | delete
@app.get("/{books/{id}}", tags=["books"],
    status_code=200, summary="...")
async def read(book_id: int): ...

async def: awaits non-blocking I/O, runs on the event
loop. Plain def: runs in a thread pool — use for blocking
libraries. Never block inside async def.
```

PARAMETER SOURCES (ANNOTATED)

```
from typing import Annotated
from fastapi import (Path, Query, Body, Header,
    Cookie, Form, File, UploadFile)
id: Annotated[int, Path(ge=1)]
q: Annotated[str | None,
    Query(max_length=50)] = None
tags: Annotated[list[str], Query()] = []
tok: Annotated[str | None, Header()] = None
sid: Annotated[str | None, Cookie()] = None
note: Annotated[str, Body(embed=True)]
user: Annotated[str, Form()]
cover: Annotated[UploadFile, File()]
f: Annotated[Filters, Query()] # model

In URL template → path; scalar → query; model → body.
Keywords: gt ge lt le min_length max_length
pattern alias deprecated. Header x_token → x=
token. Models for Query / Header / Cookie / Form (0.113–
0.115). Forms and files need python-multipart;
request: Request for the raw request.
```

PYDANTIC V2

```
from pydantic import (BaseModel, Field,
    ConfigDict, field_validator)
class BookIn(BaseModel):
    model_config = ConfigDict(
        from_attributes=True)
    title: str = Field(min_length=1)
    pages: int | None = Field(None, gt=0)
    @field_validator("title")
    @classmethod
    def strip(cls, v):
        return v.strip()
b.model_dump(exclude_unset=True)

v1          v2
.dict()      .model_dump()
.json()      .model_dump_json()
.parse_obj   .model_validate
@validator   @field_validator
orm_mode     from_attributes
class Config model_config
.schema()    model_json_schema()
```

RESPONSES & ERRORS

```
@app.post("/books", response_model=BookPublic,
    status_code=status.HTTP_201_CREATED)
def create(b: BookCreate) -> Book: ...
raise HTTPException(404, "Not found")
@app.exception_handler(RequestValidationError)
async def h(request, exc): ... # 422 default
response_class=HTMLResponse | FileResponse
| PlainTextResponse | RedirectResponse
response.set_cookie("sid", v, httponly=True)
response.headers["X-Total"] = "42"

Return annotation = response_model (0.89). Filters output
fields (no leaking password).
response_model_exclude_unset. None OK for bodiless
204 (0.117).
```

STREAMING, JSON LINES & SSE

```
StreamingResponse(gen()) # any generator
# 0.134: yield each item = JSON Lines
@app.get("/items",
    response_class=StreamingResponse)
async def items() -> AsyncIterable[Item]:
    yield Item(...)
# 0.135: Server-Sent Events
from fastapi.sse import (EventSourceResponse,
    ServerSentEvent)

@app.get("/feed",
    response_class=EventSourceResponse)
async def feed() -> AsyncIterable[
    ServerSentEvent]:
    yield ServerSentEvent(data=r, event="x")
```

DEPENDENCY INJECTION

```
def pager(skip: int = 0, limit: int = 100):
    return {"skip": skip, "limit": limit}
Pager = Annotated[dict, Depends(pager)]
# class dependency: Depends(Filters)
def get_session(): # yield dependency
    with Session(engine) as s:
        yield s
SessionDep = Annotated[Session,
    Depends(get_session)]
Depends(get_session, scope="function") # 0.121
@router.get("/", dependencies=[Depends(auth)])
FastAPI(dependencies=[Depends(audit)])
app.dependency_overrides[get_session] = fake

Cached per request; sub-dependencies nest. Exit code
after yield runs after the response (0.118);
scope="request" default.
```

SECURITY

```
oauth2 = OAuth2PasswordBearer(tokenUrl="token")
form: Annotated[OAuth2PasswordRequestForm,
    Depends()]

# PyJWT; key comes from settings, not code
tok = jwt.encode({"sub": u, "exp": exp},
    settings.secret_key, algorithm="HS256")
jwt.decode(tok, settings.secret_key,
    algorithms=["HS256"])
ph = PasswordHash.recommended() # pwdlib
ph.hash(pw); ph.verify(pw, hashed)
Security(get_user, scopes=["books:write"])
sc: SecurityScopes # .scopes, .scope_str
HTTPBasic() | HTTPBearer()
APIKeyHeader(name="X-API-Key")

401 = not authenticated (add WWW-Authenticate), 403 =
authenticated but not allowed. Security classes return 401
when credentials are missing (0.122). Compare secrets with
secrets.compare_digest.
```

MIDDLEWARE, CORS & PROXY

```
app.add_middleware(CORSMiddleware,
    allow_origins=["https://app.example.com"],
    allow_credentials=True,
    allow_methods=["*"], allow_headers=["*"])
GZIPMiddleware | TrustedHostMiddleware
FastAPIRedirectMiddleware
@app.middleware("http")
async def timing(request, call_next): ...
fastapi run --proxy-headers
--forwarded-allow-ips=<proxy ip>
FastAPI(root_path="/api") # or --root-path

Credentials + "*" origin is not allowed. Trust forwarded
headers only from your proxy.
```

SQLMODEL

```
class BookBase(SQLModel): title: str
class Book(BookBase, table=True):
    id: int | None = Field(
        default=None, primary_key=True)
class BookCreate(BookBase): ... # input
class BookPublic(BookBase): id: int # out
class BookUpdate(SQLModel):
    title: str | None = None
engine = create_engine(DATABASE_URL)
s.add(b); s.commit(); s.refresh(b)
s.exec(select(Book).offset(o).limit(n)).all()
s.get(Book, id)
b.sqlmodel_update(
    patch.model_dump(exclude_unset=True))

SQLAlchemy 2.0 underneath. create_all for prototypes,
Alembic for migrations. SQLite:
check_same_thread=False.
```

SETTINGS, LIFESPAN & BACKGROUND TASKS

```
class Settings(BaseSettings):
    model_config = SettingsConfigDict(
        env_file=".env")
    database_url: str
    secret_key: str

@lru_cache
def get_settings() -> Settings:
    return Settings()
@asynccontextmanager
async def lifespan(app: FastAPI):
    ... # startup
    yield
    ... # shutdown
app = FastAPI(lifespan=lifespan)
tasks.add_task(notify, "a@b.c") # tasks

Settings come from pydantic-settings. on_event is
superseded by lifespan (0.93). Heavy jobs → a real queue.
```

APIROUTER & LAYOUT

```
router = APIRouter(prefix="/books",
    tags=["books"],
    dependencies=[Depends(auth)])
app.include_router(router)
app/ main.py config.py dependencies.py
models/ schemas/ routers/
services/ repositories/ tests/

Router → service → repository; routers stay thin.
```

OPENAPI

```
FastAPI(title="Bookshelf", version="1.0",
    summary=..., description=...,
    docs_url="/docs", redoc_url=None,
    openapi_url="/openapi.json")
@app.get(..., operation_id="get_book",
    deprecated=True, include_in_schema=False,
    responses={404: {"model": "Err"}})
Body(openapi_examples=...) # 0.103

OpenAPI 3.1 (0.99). separate_input_output_schemas
(0.102). get_openapi() to customize.
```

WEBSOCKETS

```
@app.websocket("/ws/books/{book_id}")
async def chat(ws: WebSocket, book_id: int):
    await ws.accept()
    try:
        while True:
            msg = await ws.receive_text()
            await ws.send_text(msg)
    except WebSocketDisconnect: ...

Dependencies work; authenticate before accept(). Keep a
connection manager for broadcast.
```

STATIC, TEMPLATES & FRONTEND()

```
app.mount("/static",
    StaticFiles(directory="static"))
t = Jinja2Templates(directory="templates")
return t.TemplateResponse(
    request, "page.html", {"books": books})
app.frontend("/", directory="dist",
    fallback="index.html") # SPA, 0.138

check_dir="auto" (0.141). Dependencies on frontend()
(0.139).
```

TESTING

```
client = TestClient(app)
r = client.get("/books/1")
assert r.status_code == 200
app.dependency_overrides[get_session] = fake
async with AsyncClient(
    transport=ASGITransport(app=app),
    base_url="http://test") as ac: ...
with TestClient(app) as c: ... # lifespan

Clear overrides after each test. Schemathesis / Hypothesis
for property tests, Locust for load.
```

DEPLOYMENT

```
FROM python:3.14
WORKDIR /code
COPY requirements.txt .
RUN pip install --no-cache-dir \
    -r requirements.txt
COPY ./app /code/app
CMD ["fastapi", "run", "app/main.py",
    "--port", "8000", "--proxy-headers"]

fastapi run --workers 4 # single VM

Exec-form CMD. One process per container on Kubernetes
(replicate with pods); --workers on a single VM. Terminate
TLS at a proxy. Pin versions.
```

CHANGED SINCE OLDER TUTORIALS

0.95.0 (2023-03) Annotated recommended	0.111.0 (2024-05) fastapi CLI	0.112.0 (2024-08) fastapi[standard]	0.118.0 (2025-09) yield exit after response; pwdlib replaces passlib
0.111.1 (2024-07) PyJWT replaces python-jose	0.128.0 (2025-12) Pydantic v1 dropped	0.131.0 (2026-02) ORJSONResponse deprecated	0.132.0 (2026-02) strict_content_type
0.134.0 / 0.135.0 (2026-02/03) JSON Lines, SSE	0.138.0 (2026-06) frontend()	0.142.0 (2026-09) native OpenTelemetry	0.129.0 / 0.125.0 Python 3.9 / 3.8 dropped; fastapi-slim gone